

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF
TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Internet of Things Application Development

Final Project Report

**Smart Environmental Monitoring
System with TinyML Integration**

Advisor: Dr. Le Trong Nhan

HO CHI MINH CITY, DECEMBER 2025



Member List & Workload

No.	Full Name	Student ID	Contribution
1	Trần Minh Hiếu	2252225	100%
2	Nguyễn Đức Tài	2252225	100%
3	Nguyễn Thủy Tiên	2252806	100%

Contents

Introduction and Objectives	4
1 Core IOT Platform and Dashboard Design	8
1.1 Create Core IOT Account	8
1.1.1 Registration Process	8
1.1.2 Device Provisioning	11
1.2 Telemetry Data Transmission	13
1.3 Dashboard Implementation	15
1.3.1 Dashboard Layout Architecture	16
1.3.2 Status Indicators and Device Metrics	16
1.3.3 Remote Control Panel	16
1.3.4 Environmental Data Visualization	17
1.3.5 Alarm Management System	18
1.3.6 Geographical Fleet Tracking	18
1.3.7 Widget Configuration Best Practices	19
1.4 Summary	20
2 Hardware Implementation and Data Transmission	22
2.1 System Architecture Overview	22
2.2 Sensor and Actuator Integration	23
2.2.1 Communication Interfaces Overview	23
2.2.2 Temperature and Humidity Sensor	23
2.2.3 Light Intensity Sensor	25
2.2.4 Soil Moisture Sensor	26
2.2.5 Flame Detection Sensor	27
2.2.6 Actuator Configuration	28
2.3 Data Transmission to Core IOT Platform	29
2.3.1 MQTT Protocol Overview	29
2.3.2 Connection Establishment and Authentication	30
2.3.3 Telemetry Data Publishing	32
2.3.4 Bidirectional Communication: Remote Procedure Calls	33
2.4 Communication Protocol Analysis	34
2.4.1 Protocol Stack	34



2.4.2	Data Format and Serialization	35
2.4.3	Quality of Service Configuration	35
2.5	Thread-Safe Data Management	36
3	FreeRTOS Task Synchronization and Visual Feedback	38
3.1	Theoretical Background: Real-Time Synchronization	38
3.1.1	Semaphores and Mutexes	38
3.1.2	Message Queues	38
3.2	Task Layout Overview	39
3.2.1	Sensors and Actuators Used	39
3.3	LED Blink Control	41
3.3.1	LED Behavior Summary	41
3.3.2	Semaphore-Based State Handling	42
3.3.3	LED Task Loop	42
3.4	Humidity NeoPixel Control	43
3.4.1	Humidity-to-Color Map	43
3.4.2	State-Based Effects	43
3.4.3	NeoPixel Task Loop	44
3.5	OLED Status Display	46
3.5.1	OLED Layout Modes	46
3.5.2	State Selection Flow	48
3.5.3	Queue and Semaphore Data Flow	50
3.6	Light Intensity Processing	52
3.6.1	Signal Conditioning and Conversion	52
3.6.2	Implementation Logic	53
3.7	Soil Moisture and Automated Irrigation	53
3.7.1	Hybrid Producer-Consumer Architecture	53
3.7.2	Control Logic with Safety Timeout	54
3.8	Critical Safety Monitoring (Flame Detection)	55
3.8.1	High-Priority Event Detection	55
3.8.2	Debouncing Algorithm	55
4	Web Server Interface and Cloud Integration	57
4.1	Embedded Web Server Design	57
4.1.1	Access Point Configuration	57
4.1.2	User Interface Implementation	58
4.1.3	Web Communication Protocols	58
4.1.4	WebSocket Communication	58



4.2	Cloud Integration and Data Publishing	59
4.2.1	Station Mode Connectivity	59
4.2.2	Data Publishing to CoreIOT	59
4.2.3	Integration Improvements	60

Introduction and Objectives

Introduction

The Internet of Things (IoT) represents a transformative paradigm in modern computing, enabling physical devices to collect, exchange, and act upon data through network connectivity. As embedded systems become increasingly accessible and cloud platforms offer robust infrastructure for device management, understanding IoT architecture has become essential for engineering students. This project serves as a comprehensive learning exercise to explore the fundamental concepts and practical implementation of an IoT system.

Our project focuses on building a multi-sensor monitoring and control system using the YOLO Uno development board. The system integrates various sensors to collect environmental data and transmits this information to a cloud-based IoT platform for visualization and analysis. More importantly, the system implements bidirectional communication, allowing users to remotely control actuators connected to the device through an internet-accessible dashboard.

The core principle of IoT lies in the seamless connection between the physical and digital worlds. By developing this project, we gain hands-on experience with the complete IoT technology stack, from low-level sensor interfacing and embedded firmware development to cloud platform configuration and real-time data visualization. This holistic approach provides valuable insights into how modern connected systems operate and interact.

Objectives

The primary objective of this project is to design and implement a comprehensive environmental monitoring and control system that demonstrates the core capabilities of IoT technology. This system serves as an educational platform to understand how multiple sensors can be integrated with cloud platforms to enable real-time monitoring and remote control of physical devices. The environmental control system we build incorporates various sensors to continuously monitor environmental conditions and provides the ability to remotely control actuators such as fans, LEDs, and water pumps through an internet-connected dashboard. This project aims to showcase the complete IoT development cycle, from hardware integration and embedded firmware programming to cloud platform configuration and data visualization.

The primary objectives of this project are structured around the fundamental aspects of IoT system development:

Remote Device Control via Internet

The central objective of this project is to demonstrate remote device control through internet connectivity. We implement a system where actuators such as fans, LEDs, and water pumps can be controlled from anywhere with an internet connection. This showcases the core value proposition of IoT: the ability to monitor and control physical devices regardless of geographical location.

Sensor Integration and Data Acquisition

We integrate multiple sensors to create a comprehensive data acquisition system. The project incorporates:

- Temperature and humidity sensor (DHT11) for environmental monitoring
- Light sensor for ambient light level detection
- Soil moisture sensor for humidity measurement
- Flame sensor for fire detection scenarios

This multi-sensor approach provides experience in interfacing with different sensor types and managing concurrent data streams within a real-time operating system environment.

Cloud Platform Integration

We utilize the Core IOT platform (based on ThingsBoard) to establish device-to-cloud communication. The objectives include:

- Configuring MQTT-based telemetry transmission
- Implementing Remote Procedure Calls (RPC) for bidirectional communication
- Designing interactive dashboards for data visualization
- Understanding authentication and security mechanisms in IoT deployments

Over-the-Air (OTA) Firmware Update

A key objective is implementing OTA firmware update capability, enabling remote firmware deployment without physical access to the device. This feature is essential for IoT devices deployed in the field, allowing:

- Remote bug fixes and feature updates

- Reduced maintenance costs and downtime
- Seamless deployment of new functionality

We implement a web-based OTA interface using the ElegantOTA library, allowing firmware uploads through the device's local network interface.

Real-Time Operating System Implementation

The firmware is developed using FreeRTOS to manage multiple concurrent tasks efficiently. This provides practical experience with:

- Task scheduling and priority management
- Inter-task communication using queues and semaphores
- Thread-safe data access patterns
- Resource sharing in embedded systems

Report Structure

This report is organized into four main chapters, each covering a distinct aspect of the IoT system development:

1. Chapter 1: Core IOT Platform and Dashboard Design

This chapter covers the setup of the cloud platform, including account creation, device provisioning, telemetry transmission, and dashboard configuration with various widget types. It details the implementation of MQTT communication, RPC callbacks for remote control, and the design of interactive visualization dashboards.

2. Chapter 2: Hardware Architecture and Firmware Development

This chapter details the hardware components, circuit design, and the FreeRTOS-based firmware implementation for sensor data acquisition and actuator control. It includes the integration of multiple sensors, the real-time operating system task structure, and inter-task communication mechanisms.

3. Chapter 3: TinyML Integration

This chapter explores the integration of machine learning capabilities on the edge device for intelligent decision-making based on sensor data patterns. It covers model training, quantization, deployment, and inference on the YOLO Uno platform.



4. Chapter 4: System Integration and Testing

This chapter presents the complete system integration, testing procedures, and evaluation of the implemented IoT solution. It includes performance analysis, reliability testing, and discussion of the system's capabilities and limitations.

Chapter 1

Core IOT Platform and Dashboard Design

This chapter documents the process of setting up the Core IOT platform for our IoT project. We cover account creation, device registration, and dashboard configuration.

1.1 Create Core IOT Account

Core IOT is a cloud-based IoT platform built on the ThingsBoard framework. We selected this platform for our project due to its open-source nature and comprehensive feature set for device management and data visualization.

1.1.1 Registration Process

We accessed the Core IOT platform at <https://coreiot.io> and clicked the “Start Now” button to begin registration (Figure 1.1).

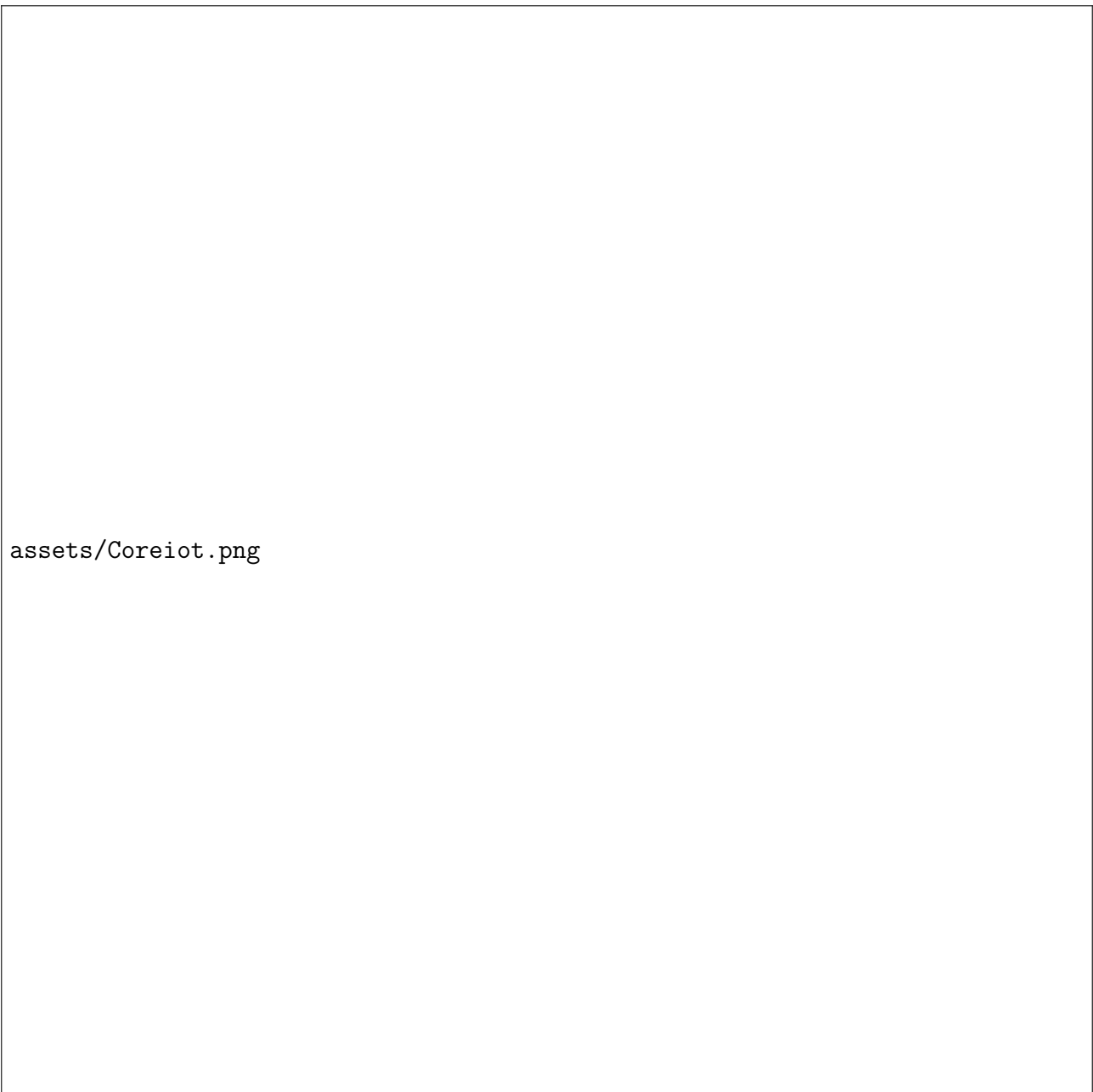


Figure 1.1: Core IOT homepage with the “Start Now” button for registration.

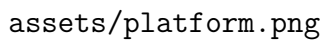
The platform offers multiple login options through OAuth providers. We chose to authenticate using a Google account for convenience (Figure 1.2).



assets/Login.png

Figure 1.2: Login interface with OAuth provider options.

After successful authentication, we were assigned the Tenant Administrator role, which provides full access to device management and dashboard creation features (Figure 1.3).



assets/platform.png

Figure 1.3: Dashboard view after login showing Tenant Administrator role.

1.1.2 Device Provisioning

With the account ready, we proceeded to register our ESP32-based device on the platform. In the “Devices” section, we created a new device and obtained an access token. This token is required for the device to authenticate when sending telemetry data via MQTT.

In our firmware, we defined global variables to store the connection credentials. These values are loaded from non-volatile storage during boot, allowing users to configure them through our web interface without modifying the source code:

```
1 // WiFi & CoreIOT Configuration
2 extern String WIFI_SSID;
3 extern String WIFI_PASS;
4 extern String CORE_IOT_TOKEN;
5 extern String CORE_IOT_SERVER;
6 extern int CORE_IOT_PORT;
```

Listing 1.1: Global configuration variables for Core IOT connection (global.h)

The connection to the Core IOT server is established using the ThingsBoard Arduino SDK. We initialize the MQTT client and ThingsBoard object as follows:

```
1 constexpr uint32_t MAX_MESSAGE_SIZE = 1024U;
2
3 WiFiClient wifiClient;
4 Arduino_MQTT_Client mqttClient(wifiClient);
5 ThingsBoard tb(mqttClient, MAX_MESSAGE_SIZE);
```

Listing 1.2: MQTT client initialization (task_core_iot.cpp)

The reconnection logic handles authentication and subscription setup. When the device connects successfully, it sends its MAC address as an attribute and subscribes to RPC callbacks:

```
1 void CORE_IOT_reconnect() {
2     if (CORE_IOT_TOKEN.isEmpty() || CORE_IOT_SERVER.isEmpty())
3         return;
4
5     if (!tb.connected()) {
6         if (!tb.connect(CORE_IOT_SERVER.c_str(),
7                         CORE_IOT_TOKEN.c_str(),
8                         CORE_IOT_PORT))
9             return;
10
11         Serial.println("[CoreIOT] Connected");
12         tb.sendAttributeData("macAddress", WiFi.macAddress().c_str());
13         tb.RPC_Subscribe(callbacks.cbegin(), callbacks.cend());
14         tb.sendAttributeData("localIp", WiFi.localIP().toString().c_str())
15     ;
16     } else {
17         tb.loop();
18     }
```

Listing 1.3: Core IOT reconnection and subscription logic

To verify the connection and data transmission capabilities, we utilized a Python script to simulate the IoT device behavior before deploying it to the actual hardware. It is critical to

note that we specifically selected the `paho-mqtt` library version 1.6.1 for this implementation, as newer versions (e.g., 2.0) introduce breaking changes in the authentication handling that are incompatible with our current configuration.

1.2 Telemetry Data Transmission

After configuring the device credentials, we developed a firmware routine to format sensor data into a JSON structure compliant with the Core IOT platform's requirements. We defined a struct to hold all sensor values in a thread-safe manner:

```
1 typedef struct {
2     float temperature;
3     float humidity;
4     float light_level;
5     float moisture_level;
6     bool flame_detected;
7     bool fan_enabled;
8     bool neoled_enabled;
9     bool water_pump_enabled;
10 } SensorData_t;
```

Listing 1.4: Sensor data structure for telemetry (`global.h`)

The main task runs in a FreeRTOS task, waiting for WiFi connectivity before attempting to connect to Core IOT. It then enters a loop that publishes telemetry data every 10 seconds:

```
1 void CORE_IOT_task(void *pvParameters) {
2     // Wait for WiFi connection
3     while (1) {
4         if (xSemaphoreTake(xBinarySemaphoreInternet, portMAX_DELAY)) {
5             Serial.println("[CoreIOT] WiFi ready");
6             break;
7         }
8         vTaskDelay(500 / portTICK_PERIOD_MS);
9     }
10
11     unsigned long lastPublish = 0;
12     const unsigned long PUBLISH_INTERVAL = 10000;
13     SensorData_t sensorData = {0};
14
15     while (1) {
16         CORE_IOT_reconnect();
17
18         if (millis() - lastPublish >= PUBLISH_INTERVAL) {
19             lastPublish = millis();
```

```
20         getSensorData(&sensorData);  // Thread-safe read
21
22         if (tb.connected()) {
23             tb.sendTelemetryData("temperature", sensorData.temperature
24         );
25             tb.sendTelemetryData("humidity", sensorData.humidity);
26             tb.sendTelemetryData("light", sensorData.light_level);
27             tb.sendTelemetryData("moisture", sensorData.moisture_level
28         );
29         }
30     }
31     vTaskDelay(500 / portTICK_PERIOD_MS);
32 }
```

Listing 1.5: Telemetry publishing routine in the main task loop

We verified the data ingestion by navigating to the “Latest Telemetry” tab within the device details page on the Core IOT portal. This interface allows us to confirm that the JSON payload is being parsed correctly and that values for temperature and humidity are updating dynamically as sent by the device.



assets/lastest_telemetry.png

Figure 1.4: The “Latest Telemetry” tab displaying real-time data received from the device.

1.3 Dashboard Implementation

One of the primary advantages of the Core IOT platform is its capability to visualize data through customizable dashboards. We created a comprehensive monitoring dashboard named *Control and Monitoring center* to serve as the central interface for real-time system oversight and actuator control. The dashboard employs a multi-panel layout that organizes information hierarchically, prioritizing critical control functions and environmental data visualization.

1.3.1 Dashboard Layout Architecture

We designed the dashboard with a responsive grid layout divided into three primary horizontal sections. The top section contains system status indicators and actuator controls, providing immediate access to the most frequently used functions. The middle section displays real-time sensor data through time-series charts and gauge widgets, enabling trend analysis and anomaly detection. The bottom section presents an alarm management table and geographical visualization, supporting incident tracking and multi-device deployment scenarios.

This hierarchical organization follows established human-computer interaction principles, placing high-priority interactive elements within the primary visual field while maintaining comprehensive data visibility through strategic use of white space and visual grouping.

1.3.2 Status Indicators and Device Metrics

The dashboard header displays two key performance indicators using card widgets. The first card shows the active device count, labeled **Devices**, which indicates the number of ESP32 devices currently connected to the platform. In our deployment, this displays a value of 1, confirming that our primary environmental monitoring device maintains an active MQTT connection.

Adjacent to the device count, we positioned a **Total** card that aggregates a custom metric across all connected devices. This counter widget can be configured to track various cumulative statistics such as total alarm count, cumulative uptime, or aggregated sensor readings. The card widgets utilize ThingsBoard's entity aggregation functions, which query the time-series database to compute values based on configurable time windows.

To the right of the status cards, we integrated a WiFi quality indicator using an icon widget. This widget dynamically changes color based on the RSSI (Received Signal Strength Indicator) value transmitted in the device telemetry. Strong signal strength displays a green icon, while degraded connectivity transitions to yellow or red, providing immediate visual feedback on network health without requiring detailed inspection of raw telemetry values.

1.3.3 Remote Control Panel

We implemented a dedicated control panel section containing three RPC toggle switches for actuator management: Water Pump, Fan, and NeoLed (RGB LED strip). Each switch widget is configured with bidirectional RPC methods to both read the current state and issue control commands.

The switch configuration employs getter and setter RPC methods. When the dashboard loads, each switch automatically invokes its corresponding getter method (`getFanStatus`, `getWaterPumpStatus`, `getLedStatus`) to synchronize the UI state with the actual device state.

This prevents desynchronization scenarios where the dashboard might display **OFF** while the device is actively driving the actuator.

When a user toggles a switch, the dashboard transmits an RPC request containing the method name and the desired boolean state. For example, clicking the Fan switch from OFF to ON triggers an RPC call to `setFanStatus` with parameter `true`. The device firmware processes this command, actuates the corresponding GPIO pin, and returns an `RPC_Response` to acknowledge successful execution.

We observed that timeout errors occasionally occur when the device is processing intensive tasks or when network latency exceeds the default RPC timeout threshold (5 seconds). In such cases, the switch widget displays a *Request timeout* modal, prompting the user to retry the command. To mitigate this issue, we optimized the RPC callback functions to execute GPIO operations quickly and defer any blocking operations to separate FreeRTOS tasks.

1.3.4 Environmental Data Visualization

The central focus of the dashboard is the environmental monitoring section, which utilizes time-series charts to visualize temperature and humidity trends. We configured a line chart widget with dual Y-axes, where temperature is plotted in degrees Celsius on the left axis and relative humidity percentage on the right axis. This dual-axis configuration enables direct visual correlation between the two variables, which is essential for identifying environmental patterns such as evaporative cooling effects.

The chart widget subscribes to the `temperature` and `humidity` telemetry keys and maintains a rolling time window of the last hour by default. Users can adjust this window through the chart's time range selector, enabling both real-time monitoring and historical analysis. We customized the chart appearance with distinct colors: red for temperature and blue for humidity, following conventional meteorological visualization standards.

Below the time-series chart, we positioned a second chart dedicated to soil moisture monitoring. This widget displays the `moisture` telemetry key, providing agricultural and plant care applications with a clear view of substrate hydration levels over time. The moisture chart uses a green color scheme and includes threshold lines at 30% (dry) and 70% (saturated) to guide irrigation decisions.

Adjacent to the charts, we implemented a large numeric display widget showing the current light level in lux. This widget is configured with the `light` telemetry key and formatted to display values with scientific notation (e.g., 65000 m², though the unit should technically be lux). The large font size makes this value readable at a glance, which is particularly useful for automated greenhouse or indoor farming applications where lighting optimization is critical.

1.3.5 Alarm Management System

We integrated a comprehensive alarm table widget in the lower section of the dashboard to track environmental anomalies and system events. The alarm system operates through rule chains configured in the ThingsBoard Rule Engine, which continuously evaluate incoming telemetry against predefined threshold conditions.

The alarm table displays several critical columns:

- **Created time:** Timestamp when the alarm was triggered, enabling chronological incident analysis
- **Originator:** The entity that generated the alarm (typically *Assignment* for our configured rule chain)
- **Type:** Classification of the alarm event (e.g., *Environment Warning*)
- **Severity:** Criticality level (Warning, Critical, or Fire Alert)
- **Status:** Current state of the alarm (Active, Acknowledged, Cleared, Unacknowledged)
- **Assignee:** User or group responsible for incident response

In our current deployment, the alarm table shows multiple *Environment Warning* entries triggered on December 3, 2025. These alarms are generated when temperature exceeds 30°C, drops below 20°C, or when humidity falls outside the 40–70% comfort range. The severity levels are color-coded: green for acknowledged alarms, yellow for warnings, and red for critical events.

We configured the alarm rules using ThingsBoard’s Rule Chain editor, which provides a visual programming interface for defining conditional logic. For example, the temperature threshold rule consists of a filter node that evaluates `temperature > 30`, connected to an action node that creates an alarm entity with severity *Warning* and type *Environment Warning*. The rule chain also sends WebSocket notifications to the dashboard, triggering real-time alert pop-ups.

The alarm table supports interactive management features. Operators can click individual alarms to view detailed information, acknowledge events to indicate they are being addressed, or assign alarms to specific team members for resolution. This functionality is particularly valuable in multi-user deployments where incident response requires coordination among maintenance personnel.

1.3.6 Geographical Fleet Tracking

At the bottom of the dashboard, we incorporated an Open Street Map widget to provide spatial context for device deployments. While our current system operates with a single de-

vice, the map widget demonstrates scalability for distributed sensor networks across multiple locations.

The map configuration requires telemetry data to include `latitude` and `longitude` fields. These coordinates can be hardcoded in the firmware for fixed installations or dynamically obtained from GPS modules for mobile deployments. Each device appears as a marker on the map, and clicking a marker displays a tooltip containing key telemetry values such as current temperature, humidity, and device status.

We customized the map widget's datasource settings to ensure proper data key mapping when multiple devices are present. By default, ThingsBoard appends numeric suffixes to duplicate data keys (e.g., `latitude_1`, `latitude_2`), but we configured explicit aliases to maintain consistent key names across all devices. This prevents widget configuration errors that would occur if the dashboard expected `latitude` but received `latitude_1`.

1.3.7 Widget Configuration Best Practices

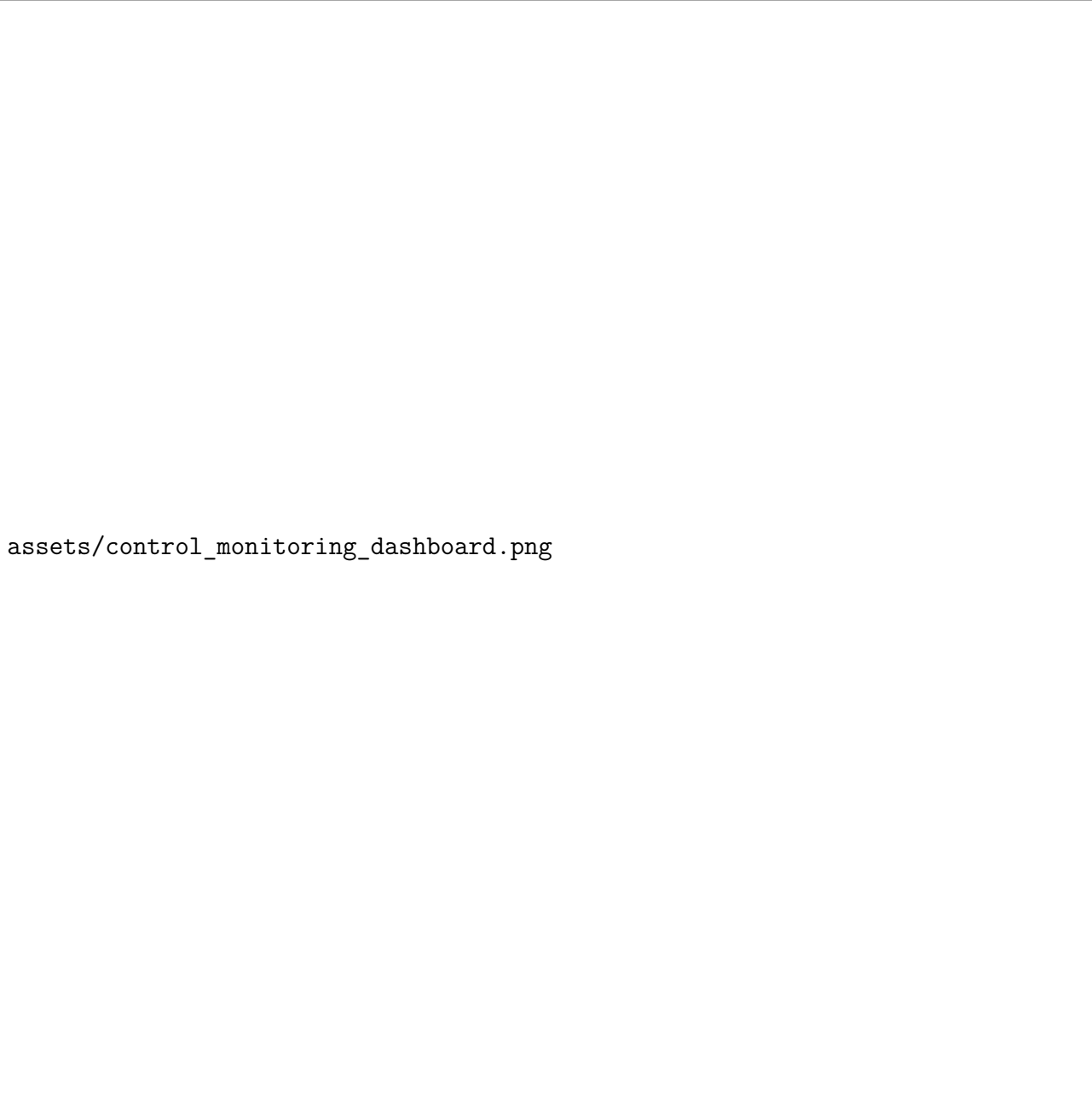
Throughout the dashboard design process, we identified several critical configuration requirements:

Data Key Consistency: The telemetry keys defined in the firmware (`temperature`, `humidity`, `light`, etc.) must exactly match the data keys configured in each widget. Case sensitivity and spelling errors will result in widgets displaying *No data* even when the device is actively transmitting.

RPC Method Naming: Switch widgets require precise RPC method names that correspond to the registered callbacks in the firmware. We standardized on a naming convention using camelCase with *set* and *get* prefixes (e.g., `setFanStatus`, `getFanStatus`).

Widget Refresh Rates: We configured chart widgets to poll for new data every 5 seconds, balancing real-time responsiveness with server load. Faster refresh rates increase database query frequency, which can impact platform performance in deployments with hundreds of devices.

Color Coding and Visual Hierarchy: We applied consistent color schemes throughout the dashboard: green for normal conditions, yellow for warnings, and red for critical alerts. This visual language enables rapid situation assessment without requiring detailed reading of all numeric values.



assets/control_monitoring_dashboard.png

Figure 1.5: Complete Control and Monitoring center dashboard showing status cards, RPC switches, time-series charts, light level display, WiFi indicator, alarm table, and map widget.

1.4 Summary

In this chapter, we successfully set up the Core IOT platform for our project. We created a Tenant Administrator account, registered our ESP32 device, and established MQTT communication for telemetry data transmission. The firmware implementation includes connection handling with automatic reconnection, periodic telemetry publishing using FreeRTOS tasks, and bidirectional RPC communication for remote device control. The dashboard was config-



ured with multiple widget types including Digital Gauges for real-time sensor readings, Charts for historical trends, RPC Switches for remote device control, and an Open Street Map for geographical visualization. These components form the foundation for the monitoring and control system that will be expanded in subsequent chapters.

Chapter 2

Hardware Implementation and Data Transmission

This chapter presents the detailed implementation of our environmental monitoring system, focusing on the hardware integration, sensor interfacing, and data transmission mechanisms. We describe how sensor data is acquired, processed on the ESP32 microcontroller, and transmitted to the Core IOT cloud platform using industry-standard protocols.

2.1 System Architecture Overview

Our system employs a distributed architecture consisting of three primary layers: the sensing layer comprising environmental sensors and actuators, the edge processing layer based on the ESP32 microcontroller running FreeRTOS, and the cloud layer utilizing the Core IOT platform for data aggregation and visualization. Figure 2.1 illustrates this three-tier architecture.

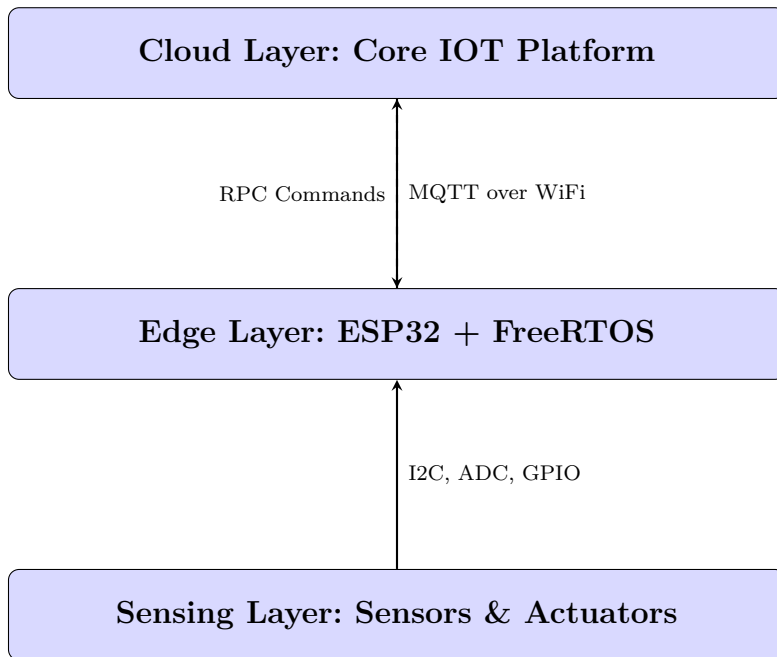


Figure 2.1: Three-tier system architecture showing data flow from sensors to cloud platform.

2.2 Sensor and Actuator Integration

We integrated multiple environmental sensors to provide comprehensive monitoring capabilities. Each sensor communicates with the ESP32 using appropriate interfaces based on its electrical characteristics and data rate requirements.

2.2.1 Communication Interfaces Overview

Inter-Integrated Circuit (I2C)

The I2C protocol is a synchronous, multi-master, multi-slave packet switched computer bus. It uses two bidirectional open-drain lines: Serial Data Line (SDA) and Serial Clock Line (SCL). We utilize I2C for the OLED display integration because it allows multiple devices to share the same bus using unique 7-bit addresses. The ESP32 acts as the master device, generating the clock signal and initiating data transfers to the SSD1306 OLED driver (address 0x3C).

Analog-to-Digital Conversion (ADC)

For analog sensors like the photoresistor and soil moisture probe, we utilize the ESP32's internal Successive Approximation Register (SAR) ADC. The ADC converts continuous analog voltage signals (0–3.3V) into discrete digital values (0–4095), enabling the microcontroller to process physical environmental parameters.

2.2.2 Temperature and Humidity Sensor

For temperature and humidity measurements, we selected the DHT11 digital sensor, which provides calibrated output through a single-wire digital interface. The DHT11 is a cost-effective sensor suitable for basic environmental monitoring applications, offering reasonable accuracy ($\pm 2^{\circ}\text{C}$ for temperature, $\pm 5\%$ for humidity) with a simple communication protocol.

The DHT11 communicates using a proprietary single-wire protocol where both data transmission and power can be shared over a single GPIO pin. We connected the sensor's data pin to GPIO 6 (labeled D4-D3 on the YoloUNO board) and utilized the Adafruit DHT library, which handles the timing-critical bit-level communication protocol automatically.

The sensor provides 8-bit resolution for both temperature (0–50°C range) and relative humidity (20–90% range). Each measurement cycle takes approximately 2 seconds due to the sensor's internal sampling rate limitation. We configured the sensor as follows:

```
1 #include <DHT.h>
2
3 #define DHTPIN 6           // GPIO 6 (D4-D3 port)
4 #define DHTTYPE DHT11     // Sensor type specification
```

```
5
6 DHT dht(DHTPIN, DHTTYPE);
```

Listing 2.1: DHT11 sensor configuration and pin definition

In our main monitoring task, we initialize the sensor and implement a polling loop with validation to ensure data integrity:

```
1 void temp_humi_oled(void *pvParameters) {
2     Serial.println("[OLED] Task started");
3
4     // Initialize DHT11 sensor
5     dht.begin();
6
7     SensorData_t sensorData = {0};
8     bool sensorReady = false;
9
10    while (1) {
11        // Read sensor values
12        float humidity = dht.readHumidity();
13        float temperature = dht.readTemperature();
14
15        // Validate readings (check for NaN and reasonable ranges)
16        bool validReading = !isnan(temperature) && !isnan(humidity) &&
17                           temperature > 0 && temperature < 80 &&
18                           humidity > 0 && humidity <= 100;
19
20        if (!validReading) {
21            vTaskDelay(pdMS_TO_TICKS(500));
22            continue; // Skip this cycle
23        }
24
25        if (!sensorReady) {
26            sensorReady = true;
27            Serial.println("[OLED] Sensor ready");
28        }
29
30        // Update shared data structure
31        sensorData.temperature = temperature;
32        sensorData.humidity = humidity;
33        sendSensorData(&sensorData);
34
35        // DHT11 requires minimum 1 second between readings
36        vTaskDelay(pdMS_TO_TICKS(1000));
37    }
38 }
```

Listing 2.2: DHT11 sensor reading with validation

The validation logic filters out erroneous readings that occasionally occur during startup or when electromagnetic interference affects the single-wire communication. We verify that values are not NaN (Not-a-Number) and fall within physically plausible ranges before accepting them. The 1-second delay between readings respects the DHT11's maximum sampling rate while providing adequate temporal resolution for environmental monitoring.

2.2.3 Light Intensity Sensor

Light intensity measurement utilizes a photoresistor (Light Dependent Resistor) coupled with an LMV358 operational amplifier to enhance signal quality. The photoresistor exhibits a nonlinear resistance characteristic that varies inversely with illumination: high resistance (several megaohms) in darkness and low resistance (hundreds of ohms) under bright light.

The LMV358 op-amp is configured as a voltage follower to buffer the signal and provide a low-impedance source for the ESP32's Analog-to-Digital Converter (ADC). The ESP32 features a 12-bit SAR ADC with a voltage range of 0–3.3V, yielding a theoretical resolution of 0.8 mV per step. However, the ADC exhibits significant nonlinearity at the extremes of its range, so we constrained our measurements to the more linear mid-range region.

We connected the sensor output to GPIO_NUM_1 (labeled A2-A1 on the YoloUNO board) and implemented an exponential mapping algorithm to convert the raw ADC values to lux units:

```
1 #define LIGHT_SENSOR_PIN GPIO_NUM_1
2 #define LUX_MIN 1.0f
3 #define LUX_MAX 65000.0f
4 #define ADC_DARK 100
5 #define ADC_BRIGHT 4000
6
7 void sensor_light_task(void *pvParameters) {
8     pinMode(LIGHT_SENSOR_PIN, INPUT);
9
10    float smoothedLux = 0;
11    const float ALPHA = 0.3f; // Smoothing coefficient
12    bool firstReading = true;
13
14    while (1) {
15        int rawValue = analogRead(LIGHT_SENSOR_PIN);
16        int constrainedADC = constrain(rawValue, ADC_DARK, ADC_BRIGHT);
17
18        // Normalize to [0, 1] range
```

```
19     float normalized = (float)(constrainedADC - ADC_DARK) /
20                           (ADC_BRIGHT - ADC_DARK);
21
22     // Exponential mapping to lux scale
23     float luxEstimate = LUX_MIN * pow(LUX_MAX / LUX_MIN, normalized);
24
25     // Exponential moving average filter
26     if (firstReading) {
27         smoothedLux = luxEstimate;
28         firstReading = false;
29     } else {
30         smoothedLux = ALPHA * luxEstimate + (1.0f - ALPHA) *
smoothedLux;
31     }
32
33     float finalLux = round(smoothedLux);
34     updateSensorField_Light(finalLux);
35
36     vTaskDelay(pdMS_TO_TICKS(1000));
37 }
38 }
```

Listing 2.3: Light sensor with exponential lux conversion and smoothing filter

The exponential mapping function accommodates the wide dynamic range of illumination levels encountered in real-world environments, spanning from moonlight (1 lux) to direct sunlight (>100,000 lux). The exponential moving average (EMA) filter reduces noise and transient fluctuations with minimal computational overhead.

2.2.4 Soil Moisture Sensor

Soil moisture monitoring employs a capacitive moisture sensor connected to GPIO_NUM_8. Unlike resistive sensors that suffer from corrosion due to electrolysis, capacitive sensors measure the dielectric constant of the soil, which correlates with water content. The sensor outputs an analog voltage proportional to moisture level.

We configured the sensor pin as an analog input and implemented a linear mapping from the 12-bit ADC range to a percentage scale:

```
1 #define MOISTURE_SENSOR_PIN GPIO_NUM_8
2
3 void sensor_moisture_task(void *pvParameters) {
4     pinMode(MOISTURE_SENSOR_PIN, INPUT);
5
6     while (1) {
```

```
7     int rawValue = analogRead(MOISTURE_SENSOR_PIN);
8
9     // Convert to percentage (inverted: high ADC = dry, low ADC = wet)
10    float moistureLevel = 100.0 - ((rawValue / 4095.0) * 100.0);
11
12    updateSensorField_Moisture(moistureLevel);
13
14    vTaskDelay(pdMS_TO_TICKS(1000));
15 }
16 }
```

Listing 2.4: Capacitive soil moisture sensor implementation

The inversion is necessary because the sensor outputs high voltage in air (dry condition) and lower voltage when immersed in water (wet condition). This configuration provides an intuitive percentage where 0% represents completely dry soil and 100% represents saturation.

2.2.5 Flame Detection Sensor

Fire detection capability is provided by an infrared flame sensor (KY-026 module) connected to GPIO pin 10. The sensor incorporates an IR photodiode tuned to detect the specific wavelength emitted by flames (approximately 760–1100 nm). The module includes an onboard comparator that generates a digital HIGH signal under normal conditions and transitions to LOW when flame radiation is detected.

To eliminate false triggers from transient electromagnetic interference or brief reflections, we implemented a debouncing algorithm requiring three consecutive positive detections before confirming a fire event:

```
1 #define FLAME_SENSOR_PIN 10
2 #define FLAME_DEBOUNCE_COUNT 3
3 #define FLAME_THRESHOLD 1500
4
5 void sensor_flame_task(void *pvParameters) {
6     pinMode(FLAME_SENSOR_PIN, INPUT);
7
8     bool confirmedFlameState = false;
9     int flameCounter = 0;
10    int noFlameCounter = 0;
11
12    while (1) {
13        int rawValue = analogRead(FLAME_SENSOR_PIN);
14        bool currentReading = (rawValue < FLAME_THRESHOLD);
15
16        // Debounce logic
```

```
17     if (currentReading) {
18         flameCounter++;
19         noFlameCounter = 0;
20         if (flameCounter >= FLAME_DEBOUNCE_COUNT && !
confirmedFlameState) {
21             confirmedFlameState = true;
22             Serial.printf("[Flame] FIRE DETECTED! (raw: %d)\n",
rawValue);
23             updateSystemState(STATE_FIRE_ALERT);
24         }
25     } else {
26         noFlameCounter++;
27         flameCounter = 0;
28         if (noFlameCounter >= FLAME_DEBOUNCE_COUNT &&
confirmedFlameState) {
29             confirmedFlameState = false;
30             Serial.printf("[Flame] Fire cleared (raw: %d)\n", rawValue
);
31         }
32     }
33
34     updateSensorField_Flame(confirmedFlameState);
35     vTaskDelay(pdMS_TO_TICKS(300));
36 }
37 }
```

Listing 2.5: Flame sensor with debouncing and state machine

This state machine approach ensures high reliability by requiring sustained detection before triggering an alarm. The 300 ms sampling interval provides rapid response while maintaining adequate noise immunity.

2.2.6 Actuator Configuration

Our system controls three actuators for automated environmental management: a cooling fan, an RGB LED strip, and a water pump. Each actuator is controlled through GPIO pins configured as digital outputs, but with distinct control logic.

The cooling fan connects to GPIO 2 and is activated in response to elevated temperature readings or manual commands from the dashboard. We drive the fan using a simple digital ON/OFF control strategy implemented through the RPC callback mechanism described in Section 2.3.

The RGB LED strip (NeoPixel) is driven by a dedicated task that modulates color and brightness based on humidity levels, as detailed in Chapter 3.

The water pump implementation is more complex, featuring a dual-mode control system:

- **Automatic Mode:** The pump operates based on soil moisture feedback with hysteresis control. It activates when moisture drops below 30% (dry threshold) and deactivates only when moisture exceeds 60% (wet threshold). This hysteresis prevents rapid cycling (chattering) of the pump relay when moisture levels fluctuate near a single threshold.
- **Manual Mode:** Users can override automatic control via the dashboard. When a manual command is received, the system enters manual mode for a fixed duration (5 minutes) before reverting to automatic operation. This safety timeout prevents the pump from running indefinitely if the user forgets to turn it off.

```
1 if (manualModeActive) {  
2     // Manual override from dashboard  
3     pumpState = manualPumpState;  
4     if (millis() > manualModeTimeout) {  
5         manualModeActive = false; // Revert to AUTO after timeout  
6     }  
7 } else {  
8     // Automatic hysteresis control  
9     if (moisture < 30.0 && !pumpState) {  
10         pumpState = true; // Turn ON  
11     }  
12     else if (moisture > 60.0 && pumpState) {  
13         pumpState = false; // Turn OFF  
14     }  
15 }  
16 digitalWrite(WATER_PUMP_PIN, pumpState ? HIGH : LOW);
```

Listing 2.6: Water pump control logic with hysteresis and manual override

2.3 Data Transmission to Core IOT Platform

After acquiring and processing sensor data locally on the ESP32, we transmit the information to the Core IOT cloud platform for storage, analysis, and visualization. This section details the connection establishment, data serialization, and transmission protocols employed.

2.3.1 MQTT Protocol Overview

We selected the Message Queuing Telemetry Transport (MQTT) protocol for all device-to-cloud communication. MQTT is an ISO standard (ISO/IEC 20922) publish-subscribe messaging protocol designed specifically for constrained devices and unreliable networks. The protocol

operates on a client-server architecture where the Core IOT platform functions as the MQTT broker and our ESP32 device acts as a client.

MQTT offers several advantages for IoT applications: minimal protocol overhead (as low as 2 bytes per message), built-in Quality of Service (QoS) levels for guaranteed delivery, and persistent session support for intermittent connectivity scenarios. The publish-subscribe model decouples data producers from consumers, allowing scalable architectures where multiple devices and services can interact without explicit peer-to-peer connections.

2.3.2 Connection Establishment and Authentication

We initialize the MQTT client using the ThingsBoard Arduino SDK, which provides a high-level abstraction over the lower-level PubSubClient library. The connection sequence begins after the WiFi task signals successful network association through a FreeRTOS binary semaphore:

```
1 constexpr uint32_t MAX_MESSAGE_SIZE = 1024U;
2
3 WiFiClient wifiClient;
4 Arduino_MQTT_Client mqttClient(wifiClient);
5 ThingsBoard tb(mqttClient, MAX_MESSAGE_SIZE);
6
7 void CORE_IOT_task(void *pvParameters) {
8     // Wait for WiFi connectivity
9     while (1) {
10         if (xSemaphoreTake(xBinarySemaphoreInternet, portMAX_DELAY)) {
11             Serial.println("[CoreIOT] WiFi ready");
12             break;
13         }
14         vTaskDelay(500 / portTICK_PERIOD_MS);
15     }
16
17     // Verify token configuration
18     if (CORE_IOT_TOKEN.isEmpty()) {
19         Serial.println("[CoreIOT] Token not configured");
20         vTaskDelete(NULL);
21         return;
22     }
23
24     unsigned long lastPublish = 0;
25     const unsigned long PUBLISH_INTERVAL = 10000; // 10 seconds
26     SensorData_t sensorData = {0};
27
28     while (1) {
29         CORE_IOT_reconnect();
```



```
30
31     if (millis() - lastPublish >= PUBLISH_INTERVAL) {
32         lastPublish = millis();
33         getSensorData(&sensorData);
34
35         if (tb.connected()) {
36             tb.sendTelemetryData("temperature", sensorData.temperature
37 );
38             tb.sendTelemetryData("humidity", sensorData.humidity);
39             tb.sendTelemetryData("light", sensorData.light_level);
40             tb.sendTelemetryData("moisture", sensorData.moisture_level
41 );
42             tb.sendTelemetryData("flame", sensorData.flame_detected ?
43 1 : 0);
44         }
45     }
46     vTaskDelay(500 / portTICK_PERIOD_MS);
47 }
```

Listing 2.7: MQTT client initialization and ThingsBoard connection

The `CORE_IOT_reconnect()` function handles both initial connection and automatic reconnection after network disruptions:

```
1 void CORE_IOT_reconnect() {
2     if (CORE_IOT_TOKEN.isEmpty() || CORE_IOT_SERVER.isEmpty())
3         return;
4
5     if (!tb.connected()) {
6         if (!tb.connect(CORE_IOT_SERVER.c_str(),
7             CORE_IOT_TOKEN.c_str(),
8             CORE_IOT_PORT))
9             return;
10
11         Serial.println("[CoreIOT] Connected");
12
13         // Send device metadata as attributes
14         tb.setAttributeData("macAddress", WiFi.macAddress().c_str());
15         tb.setAttributeData("localIp", WiFi.localIP().toString().c_str())
16 ;
17
18         // Subscribe to RPC callbacks for remote control
19         tb.RPC_Subscribe(callbacks.cbegin(), callbacks.cend());
20     } else {
21         tb.loop(); // Process incoming messages
22     }
23 }
```

```
21     }  
22 }
```

Listing 2.8: MQTT reconnection logic with authentication and subscription

Authentication occurs through a device-specific access token provisioned during device registration on the Core IOT portal. The token serves as both the MQTT username and the authorization credential, eliminating the need for separate password authentication. Upon successful connection, we transmit the device's MAC address and local IP address as attributes, enabling network diagnostics through the dashboard.

2.3.3 Telemetry Data Publishing

Telemetry data is published at a fixed interval of 10 seconds to balance timeliness with network efficiency. The ThingsBoard SDK automatically serializes the data into JSON format and publishes it to the appropriate MQTT topic. For example, when we call `tb.sendTelemetryData("temperature", 28.5)`, the SDK constructs a JSON payload similar to:

```
{"temperature": 28.5}
```

This payload is then published to the topic `v1/devices/me/telemetry`, where the broker routes it to all subscribed clients and persists it in the time-series database. Figure 2.2 shows the Core IOT dashboard displaying real-time telemetry data from our device.

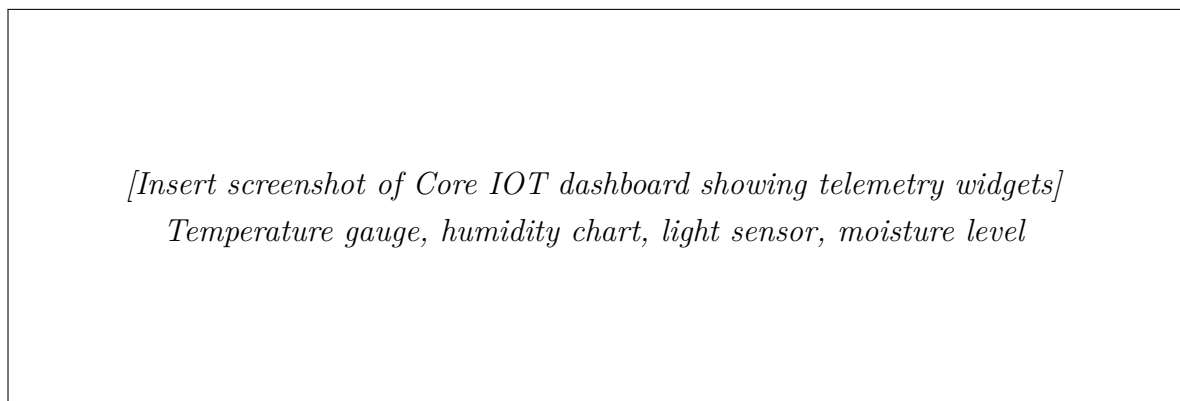


Figure 2.2: Core IOT dashboard visualizing real-time sensor telemetry data.

To verify that data is being received correctly by the platform, we accessed the *Latest Telemetry* tab within the device details page. This interface provides a real-time view of all telemetry keys and their most recent values, along with timestamps indicating when each value was last updated. Figure 2.3 demonstrates the signal verification interface.

*[Insert screenshot of Core IOT Latest Telemetry panel]
Showing temperature, humidity, light, moisture, flame keys with values and
timestamps]
Connection status: MQTT Connected*

Figure 2.3: Core IOT Latest Telemetry panel confirming successful data transmission.

This verification step is critical during development to ensure that the JSON keys defined in the firmware match exactly with the keys configured in the dashboard widgets. Any mismatch results in widgets displaying *No data* even when the device is successfully transmitting.

2.3.4 Bidirectional Communication: Remote Procedure Calls

Beyond passive telemetry transmission, we implemented bidirectional communication using ThingsBoard's Remote Procedure Call (RPC) mechanism. This allows the dashboard to invoke functions on the device, enabling remote control of actuators. We registered callback functions for each controllable actuator:

```
1 // Callback array registered with ThingsBoard SDK
2 const std::array<RPC_Callback, 6U> callbacks = {
3     RPC_Callback{"setFanStatus", setFanStatus},
4     RPC_Callback{"setLedEnabled", setLedEnabled},
5     RPC_Callback{"setWaterPumpStatus", setWaterPumpStatus},
6     RPC_Callback{"getFanStatus", getFanStatus},
7     RPC_Callback{"getLedStatus", getLedStatus},
8     RPC_Callback{"getWaterPumpStatus", getWaterPumpStatus}
9 };
10
11 // Example RPC callback for fan control
12 RPC_Response setFanStatus(const RPC_Data &data) {
13     bool newState = data;
14     fanEnabled = newState;
15
16     // Control hardware GPIO
17     pinMode(2, OUTPUT);
18     digitalWrite(2, newState ? HIGH : LOW);
```

```
19
20 // Update shared sensor data structure
21 updateSensorField_Fan(newState);
22
23 Serial.printf("[CoreIOT] Fan: %s\n", newState ? "ON" : "OFF");
24
25 // Return acknowledgment
26 return RPC_Response("setFanStatus", newState);
27 }
```

Listing 2.9: RPC callback registration and implementation

When a user toggles a switch on the dashboard, the Core IOT server publishes an RPC request to the topic `v1/devices/me/rpc/request/$request_id`. The SDK receives this message, deserializes the JSON payload, locates the corresponding callback function by name, and invokes it with the supplied parameters. The callback executes the hardware control action and returns an `RPC_Response` object, which the SDK serializes and publishes back to the server. This acknowledgment is crucial; without it, the dashboard switch would timeout and revert to its previous state.

2.4 Communication Protocol Analysis

2.4.1 Protocol Stack

Our system employs a four-layer protocol stack for device-to-cloud communication:

1. **Physical Layer:** IEEE 802.11 b/g/n WiFi operating in the 2.4 GHz ISM band
2. **Network Layer:** Internet Protocol version 4 (IPv4) with DHCP for address configuration
3. **Transport Layer:** Transmission Control Protocol (TCP) on port 1883
4. **Application Layer:** MQTT version 3.1.1 with ThingsBoard extensions

We chose TCP as the transport protocol rather than UDP to ensure reliable, ordered delivery of messages. While TCP introduces higher latency due to connection establishment and acknowledgment overhead, it guarantees that no telemetry packets are silently dropped. For our application, where sensor readings are transmitted at 10-second intervals, the additional latency (typically <100 ms on a local network) is negligible.

2.4.2 Data Format and Serialization

All telemetry data is serialized using JSON (JavaScript Object Notation), a human-readable text format that balances ease of debugging with reasonable compactness. The ThingsBoard SDK handles serialization automatically, but for clarity, we examine the structure of a typical telemetry message:

```
1 {  
2   "temperature": 28.5,  
3   "humidity": 65.2,  
4   "light": 850.0,  
5   "moisture": 45.0,  
6   "flame": 0,  
7   "fanStatus": 1,  
8   "neoLedEnabled": 1,  
9   "waterPumpStatus": 0  
10 }
```

Listing 2.10: Example JSON payload transmitted to Core IOT

The MQTT protocol wraps this JSON payload with minimal overhead: a fixed header (2 bytes), variable header containing topic information, and the payload itself. For our typical message with eight key-value pairs, the total MQTT packet size ranges from 150–200 bytes, which is well within the ESP32's available memory and the network's MTU (Maximum Transmission Unit).

2.4.3 Quality of Service Configuration

MQTT supports three QoS levels:

- **QoS 0 (At most once):** Fire-and-forget delivery with no acknowledgment
- **QoS 1 (At least once):** Acknowledged delivery, possible duplicates
- **QoS 2 (Exactly once):** Four-way handshake guarantees single delivery

We configured our telemetry transmissions with QoS 0 to minimize network traffic and broker load. For environmental monitoring, occasional message loss is acceptable because subsequent readings provide redundancy. However, for critical RPC commands controlling actuators, the ThingsBoard SDK internally uses QoS 1 to ensure delivery while avoiding the complexity of QoS 2.

2.5 Thread-Safe Data Management

A critical aspect of our implementation is ensuring thread safety when multiple FreeRTOS tasks access shared sensor data. We defined a centralized data structure to hold all sensor readings:

```
1 typedef struct {
2     float temperature;
3     float humidity;
4     float light_level;
5     float moisture_level;
6     bool flame_detected;
7     bool fan_enabled;
8     bool neoled_enabled;
9     bool water_pump_enabled;
10 } SensorData_t;
11
12 // Global primitives
13 QueueHandle_t xSensorDataQueue = NULL;
14 SemaphoreHandle_t xQueueMutex = NULL;
15
16 // Thread-safe field update function
17 void updateSensorField_Temperature(float value) {
18     if (xSemaphoreTake(xQueueMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
19         SensorData_t data;
20         if (xQueuePeek(xSensorDataQueue, &data, 0) == pdTRUE) {
21             data.temperature = value;
22             xQueueOverwrite(xSensorDataQueue, &data);
23         }
24         xSemaphoreGive(xQueueMutex);
25     }
26 }
27
28 // Thread-safe data retrieval
29 bool getSensorData(SensorData_t *data) {
30     bool success = false;
31     if (xSemaphoreTake(xQueueMutex, pdMS_TO_TICKS(100)) == pdTRUE) {
32         success = (xQueuePeek(xSensorDataQueue, data, 0) == pdTRUE);
33         xSemaphoreGive(xQueueMutex);
34     }
35     return success;
36 }
```

Listing 2.11: Sensor data structure and thread-safe access pattern

We employ a combination of FreeRTOS queue and mutex primitives to coordinate access. The queue stores a single instance of `SensorData_t` using the overwrite strategy, ensuring that sensor tasks always write the most recent value without blocking. The mutex serializes access to the queue operations themselves, preventing race conditions when multiple tasks attempt simultaneous updates.

This design pattern provides several benefits: it eliminates the risk of reading partially-updated structures (torn reads), ensures that the Core IOT task always transmits a consistent snapshot of all sensor values, and allows sensor tasks to operate independently at their natural update rates without explicit synchronization between sensors.

Chapter 3

FreeRTOS Task Synchronization and Visual Feedback

In this chapter, we describe the implementation of three real-time tasks that provide visual feedback based on sensor readings. We used FreeRTOS semaphores and queues to eliminate global variables and ensure thread-safe communication between tasks. The three tasks handle LED blinking based on temperature, NeoPixel color control based on humidity, and OLED display management with multiple states.

3.1 Theoretical Background: Real-Time Synchronization

In a multitasking environment like FreeRTOS, tasks often need to share resources or communicate with each other. Without proper synchronization, this can lead to race conditions (where two tasks modify data simultaneously) and data corruption. We employ three key FreeRTOS primitives to manage this complexity: Binary Semaphores, Mutexes, and Message Queues.

3.1.1 Binary Semaphores for Signaling

A Binary Semaphore can be visualized as a flag that can be either '0' (Empty) or '1' (Full). It is primarily used for **task synchronization** and **deferred interrupt processing**.

- **Mechanism:** When Task A completes a job (e.g., detecting a fire), it "Gives" the semaphore (sets flag to 1). Task B, which was waiting for this event, attempts to "Take" the semaphore.
- **Blocking State:** Crucially, if the semaphore is empty, Task B enters the **Blocked** state. In this state, Task B consumes **zero CPU cycles**. The FreeRTOS scheduler suspends Task B and runs other lower-priority tasks. When Task A gives the semaphore, the scheduler immediately wakes up Task B, moving it to the **Ready** state.
- **Application in Our System:** We use binary semaphores to represent system states (**Normal, Warning, Critical, Fire**). The LED and OLED tasks block on these semaphores. This is far more efficient than a "busy-wait" loop that constantly checks a global variable.

3.1.2 Mutexes for Resource Protection

A Mutex (Mutual Exclusion) is a special type of binary semaphore designed specifically for protecting shared resources, such as the I2C bus or a shared data structure.

- **Token Passing:** A mutex acts like a token. A task must "Take" the token before accessing the resource and "Give" it back when finished. If another task holds the token, the requesting task blocks until it becomes available.
- **Priority Inheritance:** Unlike standard semaphores, FreeRTOS mutexes implement *Priority Inheritance*. If a high-priority task (Task H) blocks waiting for a mutex held by a low-priority task (Task L), Task L temporarily inherits Task H's priority. This prevents ****Priority Inversion****, a scenario where a medium-priority task could preempt Task L, indefinitely blocking Task H.
- **Application in Our System:** We use `xI2CMutex` to coordinate access to the OLED display (shared between the OLED task and setup routines) and `xWaterPumpMutex` to prevent conflicts between the automatic irrigation logic and manual web control.

3.1.3 Message Queues for Inter-Task Communication

Queues provide a thread-safe way to send data structures between tasks. Unlike global variables, queues copy data by value into their own storage buffer. This ensures that the consumer receives a valid snapshot of the data even if the producer overwrites its local variable immediately after sending. This creates a robust, decoupled "Producer-Consumer" architecture.

3.2 Task Layout Overview

We implemented a producer-consumer architecture where sensor reading tasks send data to a FreeRTOS queue, and actuator control tasks read from this queue to control visual indicators. We used binary semaphores to signal system states (Normal, Warning, Critical, Fire Alert) and a mutex-protected queue to pass sensor data between tasks.

This design removes the need for global variables by storing all shared data in RTOS queues and semaphores. The queue holds the sensor readings, and semaphores signal when the system state changes. Figure 3.1 shows how tasks communicate with each other.

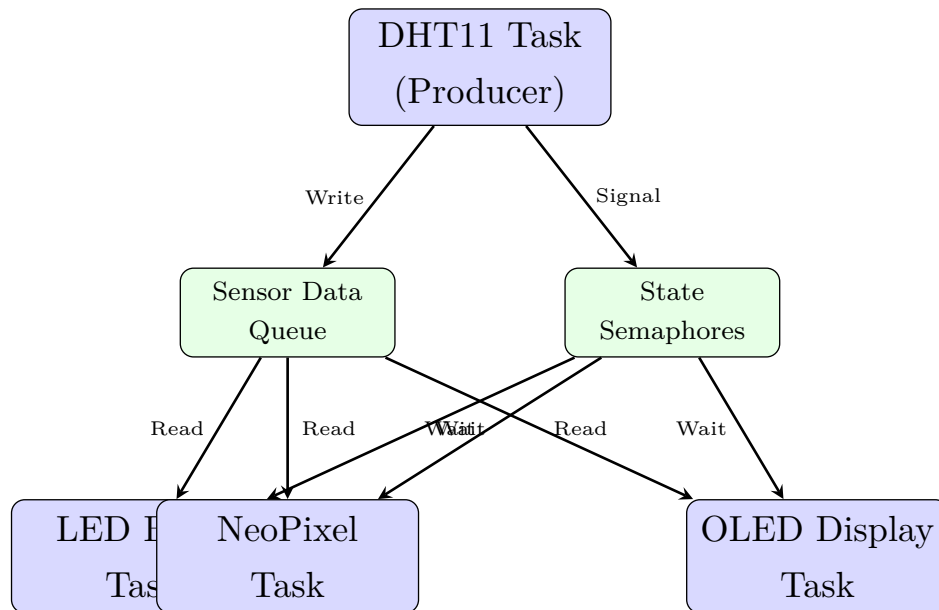


Figure 3.1: Task synchronization architecture showing producer-consumer pattern with queue and semaphore-based communication.

3.2.1 Sensors and Actuators Used

Table 3.1 summarizes the main sensors and actuators that appear in this chapter. All raw readings are stored in the shared `SensorData_t` structure and sent through the queue, while the actuator states are controlled through the LED, NeoPixel and Core IOT tasks.

Table 3.1: Overview of sensors and actuators used in the visual feedback tasks.

Type	Name	Signal	Unit	Usage in This Chapter
Sensor	DHT11 Temperature	temperature	°C	Drives LED blink modes and OLED messages; used in state evaluation (Normal/Warning/Critical/Fire).
Sensor	DHT11 Humidity	humidity	%	Controls NeoPixel color zones and OLED text; also part of state evaluation.
Sensor	Light Sensor	light_level	ADC level	Logged in the queue for future use; can be shown on OLED or dashboard. Not used in current state machine.
Sensor	Soil Moisture Sensor	moisture_level	ADC level	Logged in the queue; can be used to tune watering logic and dashboard widgets.
Sensor	Flame Sensor	flame_detected	Boolean	Immediately forces the system into Fire Alert state; affects LED, NeoPixel and OLED emergency layouts.
Actuator	Normal LED	GPIO 48	On/Off (PWM)	Indicates Normal/Warning states with different blink patterns (Task 1).
Actuator	Alert LED	GPIO 47	On/Off (PWM)	Shows Critical and Fire Alert conditions (Task 1).
Actuator	NeoPixel RGB LED	GPIO 45	Color + Brightness	Visualizes humidity ranges and system state (Task 2).
Actuator	Fan / Water Pump	Core IOT RPC	On/Off	Controlled mainly from the Core IOT dashboard; their states are still
Final Project	Report			stored in <code>SensorData.txt</code> for display and logging.

3.3 LED Blink Control

3.3.1 LED Behavior Summary

We implemented a dual-LED indicator system to show environmental conditions visually. We connected two LEDs to GPIO pins 48 (normal LED) and 47 (alert LED). The blinking pattern changes based on the system state, which depends on temperature thresholds and flame detection.

The LED task responds to four distinct system states, each triggering a unique visual pattern. This multi-state design ensures that operators can assess system status at a glance without consulting the dashboard. Table 3.2 summarizes the complete behavior specification.

Table 3.2: LED blinking behaviors mapped to system states and temperature conditions.

System State	Temperature Range	Normal LED (GPIO 48)	Alert LED (GPIO 47)	Emergency LED (GPIO 46)
Normal	20–30°C	Slow blink (ON/OFF)	OFF	5 Hz blink
Warning	<20°C or >30°C	Slow blink	OFF	5 Hz blink
Critical	>35°C	OFF	Fast blink	2 Hz blink
Fire Alert	Flame detected	OFF	Solid ON (max brightness)	—



Figure 3.2: Dual-LED indicator system showing normal LED (GPIO 48) and alert LED (GPIO 47) mounted on the YOLO Uno board.

3.3.2 Semaphore-Based State Handling

The temperature monitoring task evaluates sensor readings and signals the appropriate state using binary semaphores. We defined four binary semaphores corresponding to the four system states: `xSemaphoreNormal`, `xSemaphoreWarning`, `xSemaphoreCritical`, and `xSemaphoreFireAlert`.

When a state transition occurs, the monitoring task first clears all state semaphores by taking them with a zero timeout, then gives only the semaphore corresponding to the new state. This ensures that exactly one state is active at any time, preventing ambiguous conditions. The LED task can then check which semaphore is available to determine the current state.

Figure 3.3.2 illustrates the state machine governing LED behavior:

3.3.3 LED Task Loop

The LED task operates in an infinite loop, continuously monitoring the sensor queue and system state. The core logic follows this pseudocode structure:

```
TASK led_blinky():
    Initialize GPIO pins 48 and 47 as outputs
    Initialize local variables (blinkState, sensorData)

    WHILE TRUE:
        // Non-destructive queue read
        Peek sensor data from xSensorDataQueue
        Read current system state (thread-safe)

        IF state == FIRE_ALERT OR flame_detected:
            Set Alert LED to maximum brightness (GPIO 47 = HIGH)
            Set Normal LED off (GPIO 48 = LOW)
            Delay 100ms
        ELSE:
            Set Alert LED off (GPIO 47 = LOW)
            Toggle blinkState
            Set Normal LED according to blinkState
            Delay 500ms
    END WHILE
END TASK
```

The task uses `xQueuePeek()` to read sensor data without removing it from the queue, so multiple tasks can read the same data. The `getSystemState()` function safely reads the current

state using a mutex. We set the blink period to 500 ms, which is fast enough to catch attention but not too fast to be annoying.

The semaphore-based state checking follows this logic: the task reads sensor data from the queue using a non-destructive peek operation, then calls `getSystemState()` which internally checks which state semaphore is available. If the state is `STATE_FIRE_ALERT`, the alert LED is set to maximum brightness and the normal LED is turned off. Otherwise, the normal LED blinks at 500 ms intervals while the alert LED remains off.

Role of Mutex in State Access: Although the LED task runs periodically using `vTaskDelay` to maintain the blink frequency, it relies on the `xStateMutex` (implied within `getSystemState`) to safely read the shared system state. This ensures that the task never reads a partially updated state or encounters a race condition if the sensor task is simultaneously transitioning the system between Normal and Warning states.

3.4 Humidity NeoPixel Control

3.4.1 Humidity-to-Color Map

We implemented a NeoPixel (WS2812B RGB LED) controller that changes color based on humidity levels. We chose colors that are easy to understand: green for good conditions, red for problems, and other colors for intermediate states.

Table 3.3 defines the complete humidity-to-color mapping specification:

Table 3.3: NeoPixel color mapping for different humidity ranges with RGB values.

Humidity Range	Condition	State	Color	RGB Values
<30%	Too Dry	Critical	Red	(255, 0, 0)
30–40%	Dry	Warning	Orange	(255, 100, 0)
40–60%	Ideal	Normal	Green	(0, 255, 0)
60–70%	Acceptable	Normal	Cyan	(0, 200, 255)
>70%	Too Humid	Warning	Purple	(150, 0, 255)

We created five color zones to give more detailed feedback. The 40–60% range shows green (ideal conditions), while other ranges show cyan (acceptable), orange (dry warning), or purple (too humid). If humidity drops below 30%, we show red to indicate a critical condition.

3.4.2 State-Based Effects

The NeoPixel behavior changes based on the system state. In Normal state, it shows a solid color matching the humidity level. In Warning or Critical states, it blinks to catch attention.

During Fire Alert, it shows solid bright red at maximum brightness, overriding all other logic.

Figure 3.3 presents the decision flow implemented in the NeoPixel task:

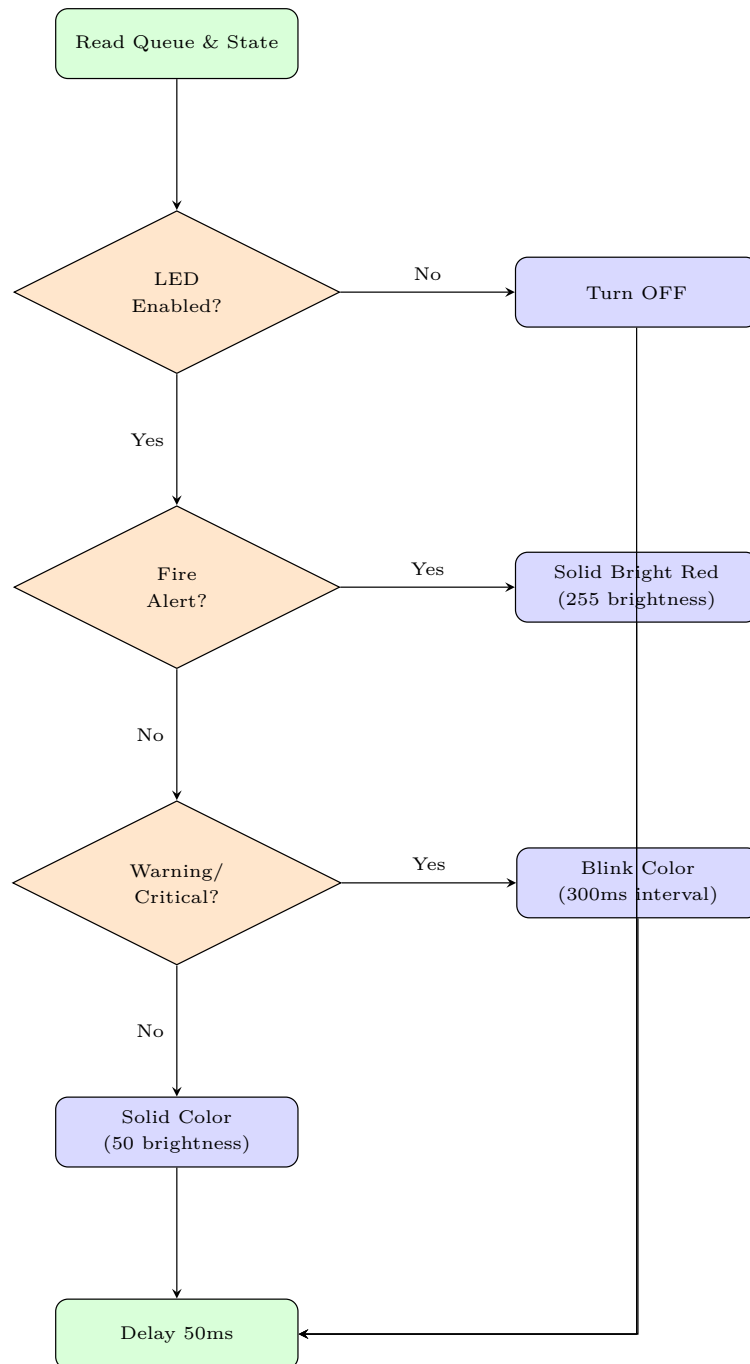


Figure 3.3: NeoPixel control flow showing decision logic based on LED enable status and system state.

3.4.3 NeoPixel Task Loop

The NeoPixel task implements a hierarchical priority system where fire alerts override all other conditions. The pseudocode structure demonstrates this layered decision logic:

```
FUNCTION getColorByHumidity(humidity):  
    IF humidity < 30: RETURN Red(255,0,0)  
    ELSE IF humidity < 40: RETURN Orange(255,100,0)  
    ELSE IF humidity <= 60: RETURN Green(0,255,0)  
    ELSE IF humidity <= 70: RETURN Cyan(0,200,255)  
    ELSE: RETURN Purple(150,0,255)  
END FUNCTION  
  
TASK neo_blinky():  
    Initialize NeoPixel strip on GPIO 45  
    Set default brightness to 50  
  
    WHILE TRUE:  
        Read sensor data from queue  
        Read current system state  
  
        IF LED disabled (via RPC control):  
            Turn off LED  
            CONTINUE  
        END IF  
  
        IF state == FIRE_ALERT OR flame detected:  
            Set brightness to 255 (maximum)  
            Display solid bright red  
        ELSE IF state == WARNING OR CRITICAL:  
            Set brightness based on severity (70 or 100)  
            Get color from humidity mapping  
            Blink color at 300ms intervals  
        ELSE (Normal state):  
            Set brightness to 50 (power saving)  
            Get color from humidity mapping  
            Display solid color  
        END IF
```



```
        Delay 50ms  
    END WHILE  
END TASK
```

The color selection function checks humidity ranges to pick the right color. We adjust brightness to save power during normal operation (brightness 50) and make alerts more visible (brightness 70 for warnings, 100 for critical). This makes critical states stand out even when using the same color.

The semaphore-based state checking works as follows: the task first checks if the LED is enabled via RPC control by reading from the sensor data queue. If disabled, the LED remains off. Otherwise, it calls `getSystemState()` to determine the current system state. Based on the state semaphore, the task selects the appropriate behavior: fire alert displays solid bright red at maximum brightness, warning/critical states blink the humidity-based color at 300 ms intervals, and normal state displays a solid color at reduced brightness.

3.5 OLED Status Display

3.5.1 OLED Layout Modes

We implemented an OLED display (SSD1306, 128x64 pixels) that shows environmental data with different layouts based on the system state. In Normal state, it shows detailed temperature and humidity readings. In Warning or Critical states, it highlights the problem. In Fire Alert, it shows an emergency message.

Table 3.4 enumerates the display states and their visual characteristics:

Table 3.4: OLED display states with corresponding visual elements and trigger conditions.

Display State	Trigger Condition	Visual Elements	Information Displayed
Normal	20–30°C and 40–70% humidity	Clean layout, comfort status (IDEAL/-GOOD), smiley icon	Full temp/humidity values, status text
Warning	Temperature or humidity outside normal range	Blinking border (500ms), warning message (HOT/-COLD/DRY/HUMID), exclamation icon	Abbreviated readings, specific warning reason
Critical	Temperature >35°C or humidity <30%	Inverted header (flashing), action message, large exclamation	Temperature, humidity, corrective action text
Fire Alert	Flame detected	Full screen flash (white/black), FIRE!! text, EVACUATE NOW message	Emergency alert only, no sensor data



Figure 3.4: OLED screen on the YOLO Uno board showing temperature and humidity with different layouts for each state.

3.5.2 State Selection Flow

The OLED task checks which state semaphore is available to decide which display function to call. Instead of constantly checking variables, the semaphore approach lets the task wait until the state changes, which saves CPU time.

We created separate functions for each display state: `drawNormalDisplay()`, `drawWarningDisplay()`, `drawCriticalDisplay()`, and `drawFireAlertDisplay()`. This makes the code easier to maintain and test.

Figure 3.5 illustrates the OLED rendering pipeline:

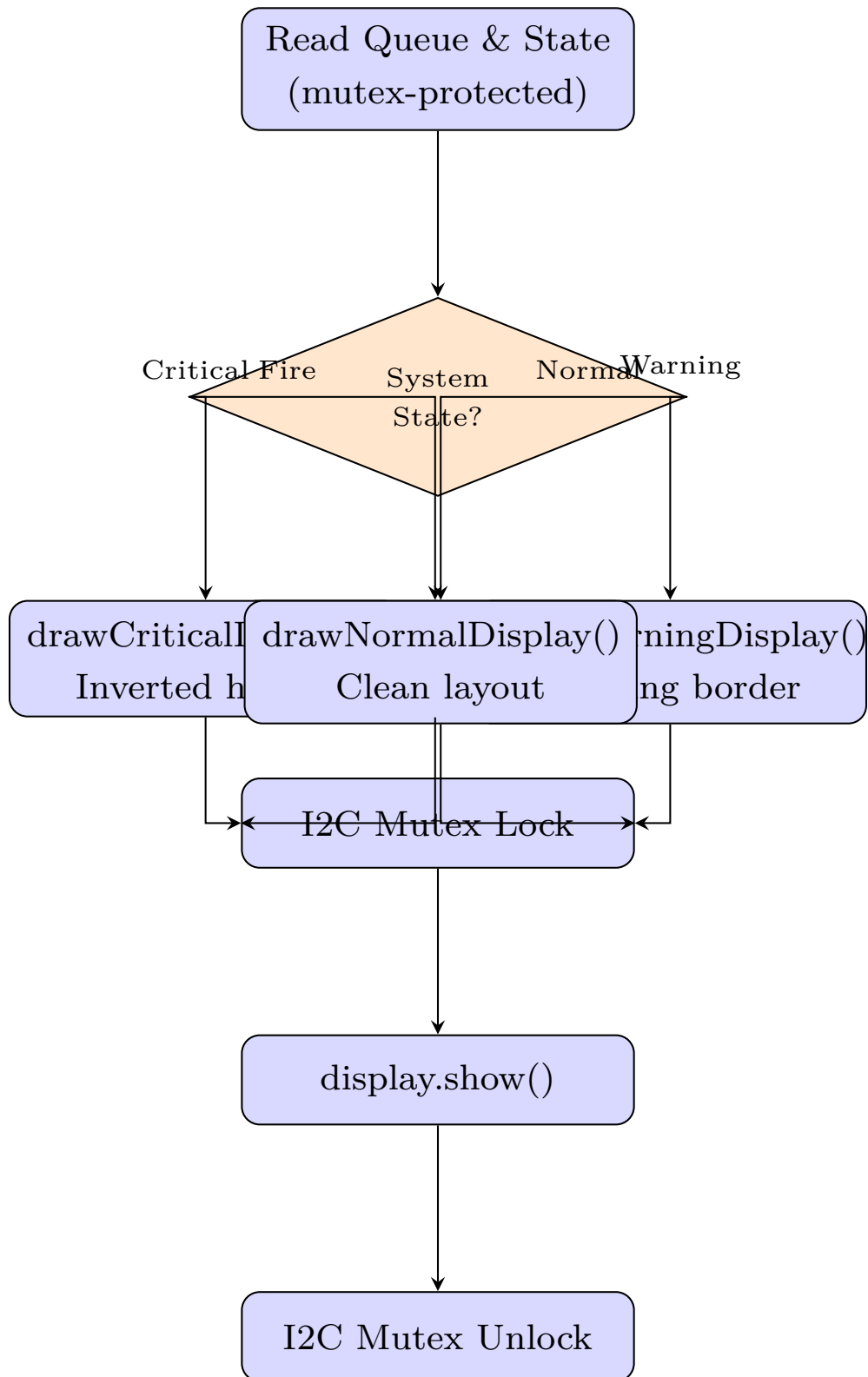


Figure 3.5: OLED rendering pipeline showing state-based function selection and I2C mutex protection.

3.5.3 Queue and Semaphore Data Flow

One of the main requirements was to remove all global variables and use RTOS primitives instead. We did this by putting all sensor readings into a `SensorData_t` structure that goes through the queue, and using semaphores for state information.

We removed global variables in three ways:

1. **Data in Queue:** All sensor values are stored in the `SensorData_t` struct in the queue, so we don't need separate global variables for temperature, humidity, etc.
2. **State with Semaphores:** We use binary semaphores to signal system states instead of global state variables
3. **Task Communication:** Tasks talk to each other through queues and semaphores, not shared memory, which makes it thread-safe

The OLED task demonstrates this pattern through its main loop structure:

```
TASK temp_humi_oled():
    Initialize OLED display (with I2C mutex protection)
    Initialize DHT11 sensor

    WHILE TRUE:
        Read temperature and humidity from DHT11
        Validate readings

        // Pack data into struct (no global variables)
        sensorData.temperature = temperature
        sensorData.humidity = humidity

        // Evaluate state using pure function
        newState = evaluateSystemState(temp, humidity, flame)

        // Update semaphores (thread-safe)
        updateSystemState(newState)

        // Send to queue for other tasks
        sendSensorData(sensorData)

        // Render display based on state
```

```
    Acquire I2C mutex
    currentState = getSystemState()
    SWITCH currentState:
        CASE FIRE_ALERT: drawFireAlertDisplay()
        CASE CRITICAL: drawCriticalDisplay()
        CASE WARNING: drawWarningDisplay()
        CASE NORMAL: drawNormalDisplay()
    Release I2C mutex

    // Adaptive delay based on urgency
    IF state >= WARNING: Delay 300ms
    ELSE: Delay 1000ms
END WHILE
END TASK
```

The state evaluation is implemented as a pure function with no side effects:

```
FUNCTION evaluateSystemState(temp, humidity, flame):
    IF flame: RETURN FIRE_ALERT
    IF temp > 35 OR humidity < 30: RETURN CRITICAL
    IF temp < 20 OR temp > 30 OR humidity < 40 OR humidity > 70:
        RETURN WARNING
    RETURN NORMAL
END FUNCTION
```

The state update function handles semaphore signaling with mutex protection, ensuring atomic state transitions:

```
FUNCTION updateSystemState(newState):
    Acquire state mutex
    IF newState != currentState:
        // Clear all semaphores atomically
        Take all state semaphores (timeout = 0)

        // Give only the active state semaphore
        SWITCH newState:
            CASE NORMAL: Give xSemaphoreNormal
            CASE WARNING: Give xSemaphoreWarning
            CASE CRITICAL: Give xSemaphoreCritical
```

```
CASE FIRE_ALERT: Give xSemaphoreFireAlert
```

```
    currentState = newState
```

```
    Release state mutex
```

```
END FUNCTION
```

This architecture completely eliminates global variable dependencies by using FreeRTOS queues for data passing and semaphores for state signaling, achieving deterministic thread-safe inter-task communication without shared memory vulnerabilities.

The OLED task works like this: it reads temperature and humidity from the DHT sensor, checks if the readings are valid, and puts them into a `SensorData_t` structure. This structure is sent to the queue so other tasks can read it, avoiding global variables.

To determine the system state, we call `evaluateSystemState()` which takes sensor readings and returns the state based on thresholds. This function doesn't use any global variables. Then we call `updateSystemState()` to update the semaphores.

The `updateSystemState()` function works atomically: it first takes a mutex, then clears all state semaphores by taking them with zero timeout, and finally gives only the semaphore for the new state. This way, only one state semaphore is available at a time, so other tasks can check which state is active.

For displaying, the task takes the I2C mutex before writing to the OLED, calls `getSystemState()` to check which state semaphore is available, and then calls the right display function. We use different delays: 300 ms during warnings/critical states for faster updates, and 1000 ms during normal operation to save CPU.

3.6 Light Intensity Processing

3.6.1 Signal Conditioning and Conversion

The light sensor task functions as a pure producer in our architecture. It acquires analog voltage readings from the photoresistor circuit and converts them into meaningful lux units. The conversion process is non-trivial due to the logarithmic response of the human eye and the sensor's characteristics.

We implemented a digital signal processing pipeline that includes:

- **Input Constraining:** Clamping raw ADC values to the linear region of the sensor.
- **Exponential Mapping:** Converting linear ADC steps to a logarithmic lux scale.
- **Smoothing Filter:** Applying an Exponential Moving Average (EMA) to reduce measurement noise.

3.6.2 Implementation Logic

The task operates independently, publishing processed values to the shared sensor queue.

```
TASK sensor_light_task():
    Initialize ADC on GPIO 1
    Initialize smoothedLux = 0
    ALPHA = 0.3 // Smoothing factor

    WHILE TRUE:
        rawADC = analogRead(LIGHT_SENSOR_PIN)
        constrained = constrain(rawADC, 100, 4000)

        // Normalize to 0.0 - 1.0
        normalized = (constrained - 100) / 3900.0

        // Exponential mapping: 1 lux to 65000 lux
        luxEstimate = 1.0 * pow(65000.0, normalized)

        // Low-pass filter
        smoothedLux = (ALPHA * luxEstimate) + ((1 - ALPHA) * smoothedLux)

        // Update shared data
        updateSensorField_Light(smoothedLux)

        Delay 1000ms
    END WHILE
END TASK
```

3.7 Soil Moisture and Automated Irrigation

3.7.1 Hybrid Producer-Consumer Architecture

The water pump task represents a hybrid producer-consumer model. It consumes moisture data from the shared queue to make control decisions (Consumer role) and produces pump status updates back to the system (Producer role).

This task manages the water pump actuator using a robust hysteresis control algorithm to prevent rapid switching cycles. It also integrates a manual override mechanism controlled

via the dashboard, which requires careful synchronization using a mutex to prevent conflict between automatic logic and user commands.

3.7.2 Control Logic with Safety Timeout

The implementation prioritizes user commands but includes a safety timeout to revert to automatic mode, preventing flooding accidents.

```
TASK sensor_water_pump_task():
    Initialize Pump GPIO as OUTPUT
    Create xWaterPumpMutex

    WHILE TRUE:
        // Consume latest sensor data
        Get sensorData from Queue

        Acquire xWaterPumpMutex

        // Check manual mode expiration
        IF manualModeActive AND (millis() > manualTimeout):
            manualModeActive = FALSE

        IF manualModeActive:
            pumpState = manualState
        ELSE:
            // Auto mode with Hysteresis
            IF moisture < 30.0 AND pumpState == OFF:
                pumpState = ON
            ELSE IF moisture > 60.0 AND pumpState == ON:
                pumpState = OFF

        Release xWaterPumpMutex

        // Actuate hardware
        digitalWrite(PUMP_PIN, pumpState)

        // Update system status
        updateSensorField_WaterPump(pumpState)
```

```
        Delay 500ms
    END WHILE
END TASK
```

Mutex for Conflict Prevention: The `xWaterPumpMutex` plays a vital role in this task. Since the water pump can be controlled by two independent sources—the automatic logic in this task and the manual override commands from the Web Server task (via RPC)—there is a risk of conflicting write operations. The mutex ensures mutual exclusion: if the Web Server is updating the manual mode status, the automatic task must wait until that operation is complete before it can read or write the pump state. This guarantees data integrity and prevents "glitchy" actuator behavior.

3.8 Critical Safety Monitoring (Flame Detection)

3.8.1 High-Priority Event Detection

The flame detection task is assigned the highest priority in our system (Priority 4) to ensure immediate response to fire hazards. Unlike other monitoring tasks that report data periodically, this task operates as a watchdog that can trigger a system-wide state transition to `STATE_FIRE_ALERT`.

To prevent false alarms caused by transient infrared noise (e.g., remote controls or sunlight glint), we implemented a debouncing state machine. This requires multiple consecutive positive readings before confirming a fire event.

3.8.2 Debouncing Algorithm

The logic ensures reliability by validating the signal stability over a short time window.

```
TASK sensor_flame_task():
    Initialize Flame Sensor GPIO 10
    flameCounter = 0
    confirmedState = FALSE

    WHILE TRUE:
        rawVal = analogRead(FLAME_PIN)
        isFlame = (rawVal < THRESHOLD)

        IF isFlame:
```

```
        flameCounter++
    IF flameCounter >= 3 AND !confirmedState:
        confirmedState = TRUE
        // Critical State Transition
        updateSystemState(STATE_FIRE_ALERT)
ELSE:
    flameCounter = 0
    IF confirmedState:
        confirmedState = FALSE
        // Clear Alert
        updateSystemState(STATE_NORMAL)

    // Update shared data
    updateSensorField_Flame(confirmedState)

    Delay 300ms // Fast sampling
END WHILE
END TASK
```

Chapter 4

Web Server Interface and Cloud Integration

This chapter details the implementation of the user interface and cloud connectivity features. We designed a dual-mode network architecture where the ESP32 operates simultaneously as an Access Point (AP) for local control and a Station (STA) for cloud data publishing. This hybrid approach ensures that the device remains controllable locally even when internet connectivity is lost.

4.1 Embedded Web Server Design

We developed a responsive web interface hosted directly on the ESP32's flash memory using the LittleFS file system. The web server is built on the `ESPAsyncWebServer` library, which provides asynchronous request handling to maintain system responsiveness even under heavy network load.

4.1.1 Access Point Configuration

To ensure the device is always accessible, we configured the ESP32 to broadcast its own WiFi network. This allows users to connect directly to the device without needing an external router.

- **SSID:** `YoloUNO_AP`
- **IP Address:** `192.168.4.1` (Static)
- **Security:** WPA2-PSK (Password protected)
- **Mode:** `WIFI_AP_STA` (Simultaneous AP + Station)

The network initialization logic in `task_wifi.cpp` ensures that the AP interface remains active even while the device attempts to connect to an external WiFi network. This "AP-first" strategy prevents the device from becoming unreachable if the home WiFi credentials change or the router goes offline.

4.1.2 User Interface Implementation

The user interface is implemented using standard HTML5, CSS3, and JavaScript, served as static files from the device's filesystem. This separation of concerns allows us to update the UI look-and-feel without recompiling the firmware.

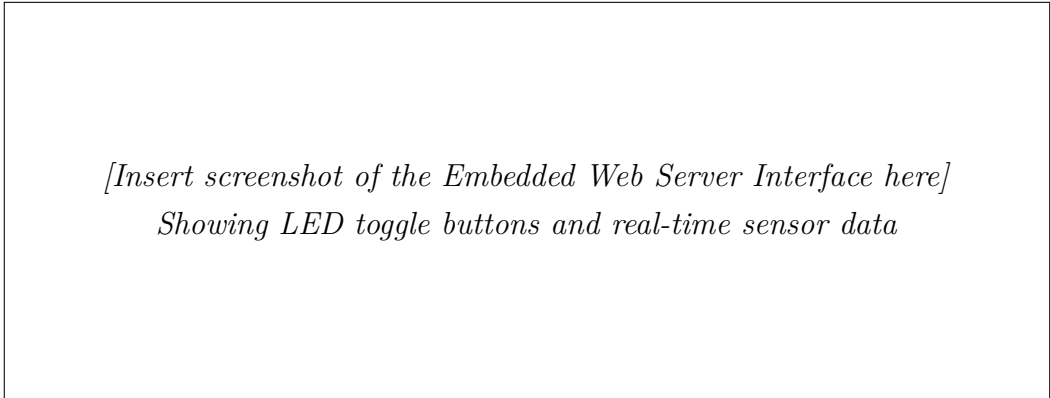
The interface features a dashboard layout with two primary control sections:

1. Device Control Panel:

- **LED Control:** A toggle switch to control the onboard LED (GPIO 48).
- **Fan/Pump Control:** A secondary toggle for the high-power actuator (GPIO 2 or Pump).

2. Real-time Status:

- **Sensor Readings:** Live updates of temperature, humidity, and light levels.
- **Connection Status:** Indicators for WiFi signal strength and Cloud connectivity.



*[Insert screenshot of the Embedded Web Server Interface here]
Showing LED toggle buttons and real-time sensor data*

Figure 4.1: Embedded Web Server interface showing control buttons and sensor status.

4.1.3 Web Communication Protocols

HTTP vs. WebSocket

Traditional web servers use the HTTP protocol, which follows a request-response model. The client requests data, the server responds, and the connection is closed. This is inefficient for real-time control because the client must constantly poll the server to get updates.

For our control interface, we chose the **WebSocket** protocol. WebSocket provides a full-duplex communication channel over a single TCP connection. Once established, the server can push data to the client immediately without waiting for a request. This significantly reduces latency and network overhead, making it ideal for controlling actuators (like LEDs) and displaying live sensor data.

4.1.4 WebSocket Communication

To achieve real-time performance without page reloads, we utilized the WebSocket protocol. Unlike traditional HTTP polling, WebSocket establishes a persistent, full-duplex connection between the browser and the ESP32.

```
1 void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client,
2             AwsEventType type, void *arg, uint8_t *data, size_t len) {
3     if (type == WS_EVT_DATA) {
4         AwsFrameInfo *info = (AwsFrameInfo *)arg;
5         if (info->opcode == WS_TEXT) {
6             String message = String((char *)data).substring(0, len);
7
8             // Parse JSON command
9             // Example: {"action":"toggleLed", "value":true}
10            handleWebSocketMessage(message);
11        }
12    }
13 }
```

Listing 4.1: WebSocket event handler for real-time control

When a user clicks a button on the web interface, a JSON message is sent over the WebSocket. The ESP32 parses this message and triggers the corresponding FreeRTOS task (e.g., giving a semaphore to the LED task). Conversely, when sensor values change, the ESP32 broadcasts the new data to all connected clients immediately.

[Insert diagram of WebSocket Communication Flow here]
Client -> JSON Command -> ESP32 -> Task Notification

Figure 4.2: WebSocket communication flow for real-time device control.

4.2 Cloud Integration and Data Publishing

While the local web server provides immediate control, long-term data storage and remote access are handled by the CoreIOT cloud platform.

4.2.1 Station Mode Connectivity

The device connects to the local WiFi network in Station (STA) mode. We implemented a robust reconnection state machine that:

1. Attempts to connect to the configured WiFi SSID.
2. If connection fails, it waits for a backoff period before retrying.
3. Periodically checks connection health and triggers reconnection if the link is dropped.
4. Synchronizes system time via NTP (Network Time Protocol) upon successful connection.

Crucially, the reconnection logic runs in a low-priority background task to avoid blocking critical sensor processing.

4.2.2 Data Publishing to CoreIOT

We utilize the MQTT protocol to publish sensor telemetry to the CoreIOT server (`app.coreiot.io`). The publishing logic is encapsulated in the `CORE_IOT_task`, which aggregates data from the shared sensor queue and transmits it securely.

Authentication Configuration: The connection is authenticated using a unique Device Token. This token acts as the MQTT username, ensuring that data is routed to the correct device dashboard in the cloud.

```
1 // CoreIOT Configuration
2 #define CORE_IOT_SERVER "app.coreiot.io"
3 #define CORE_IOT_PORT 1883
4 #define DEVICE_TOKEN "YOUR_DEVICE_TOKEN_HERE" // Replaced with actual
   token
5
6 void publishTelemetry() {
7     if (tb.connected()) {
8         // Aggregate latest readings
9         SensorData_t data;
10        getSensorData(&data);
11
12        // Publish to cloud
13        tb.sendTelemetryData("temperature", data.temperature);
14        tb.sendTelemetryData("humidity", data.humidity);
15        tb.sendTelemetryData("light", data.light_level);
16
17        Serial.println("[Cloud] Data published");
18    }
19 }
```

Listing 4.2: CoreIOT authentication and data publishing

4.2.3 Integration Improvements

Building upon the initial setup described in Chapter 1, we enhanced the cloud integration with several reliability features:

- **Data Buffering:** Sensor readings are buffered in the FreeRTOS queue, ensuring that the latest valid reading is always available for publishing, even if the sensor task is temporarily busy.
- **Connection Resilience:** The system automatically handles MQTT broker disconnections without losing state. It attempts to reconnect to the cloud independently of the WiFi connection status.
- **Traffic Optimization:** Telemetry is published at a fixed interval (10 seconds) rather than on every sensor change, reducing bandwidth usage and complying with CoreIOT rate limits.

This dual-interface approach—local Web Server for low-latency control and CoreIOT Cloud for historical analysis—provides a comprehensive user experience that combines the best aspects of edge and cloud computing.